Type Traits and Compile-time C++

Ask questions about types and compute values while compiling: `<type_traits>`, `enable_if`, `if constexpr`, `constexpr` and template metaprogramming.

Type traits

A **type trait** answers a question about a type at compile time (`::value` / `_v`) or transforms one (`::type` / `_t`).

```
#include <type_traits>

static_assert(std::is_integral_v<int>);                        // query: true
static_assert(!std::is_integral_v<double>);
static_assert(std::is_same_v<int, std::remove_reference_t<int&>>);   // transform, then compare

template <typename T>
T multiply(T a, T b) {
    static_assert(std::is_arithmetic_v<T>, "multiply needs a number type");
    return a * b;
}
// multiply(std::string("a"), std::string("b"));   // compile error with YOUR message
```

Explore, especially the references and `const` rows, where people guess wrong:

EXPLORER **What do the standard traits say about a type?**

Type T: int

TRAIT	STD::TRAIT_V<T>
`std::is_integral_v`	**true**
`std::is_floating_point_v`	**false**
`std::is_arithmetic_v`	**true**
`std::is_pointer_v`	**false**
`std::is_reference_v`	**false**
`std::is_const_v`	**false**
`std::is_class_v`	**false**
`std::is_enum_v`	**false**
`std::is_trivially_copyable_v`	**true**

KEY POINT `static_assert`: compile-time, free. `assert`: run-time, gone when `NDEBUG` is defined.

READ MORE

Programming: Principles and Practice (Stroustrup)

- 18.1.3 Concepts (PDF pages 531 to 533)
- 19.4 and 21.1 Generalizing Vector again; standard algorithms (PDF pages 573 to 578 and 617 to 622)

`enable_if` and SFINAE

SFINAE: an invalid substitution drops that overload instead of erroring. `std::enable_if` uses it to switch overloads by a condition.

```
template <typename T, std::enable_if_t<std::is_integral_v<T>, bool> = true>
void print(const T& v) { std::cout << "integral: " << v << '\n'; }

template <typename T, std::enable_if_t<!std::is_integral_v<T>, bool> = true>
void print(const T& v) { std::cout << "other: " << v << '\n'; }

print(42);      // first overload
print(2.5);     // second overload: for double the first overload's enable_if
has no ::type, so it is discarded
```

Noisy syntax, poor errors. Better tools:

TOOL	STANDARD	LOOKS LIKE
`if constexpr`	C++17	Branch inside *one* function; the untaken branch is not even compiled for that type
Concepts / `requires`	C++20	`template <std::integral T> void print(T v);`

```
template <typename T>
void print(const T& v) {
    if constexpr (std::is_integral_v<T>) std::cout << "integral: " << v <<
'\n';
    else                                 std::cout << "other: " << v << '\n';
}

template <std::integral T>        // C++20: constrained template, readable
errors
T twice(T v) { return v * 2; }
```

> PLAIN IF IS NOT ENOUGH With `if (is_integral_v<T>)` **both** branches must compile for every `T`. `if constexpr` discards the false one.

READ MORE

Effective Modern C++ (Meyers)

- Items 26 to 27, Overloading on universal references and its alternatives (tag dispatch, enable_if) (PDF pages 195 to 215)

Concepts and `requires` (C++20)

A **concept** is a named, compile-time condition on a type. It replaces `enable_if` with something you can read, and it turns a page of substitution errors into one clear message at the call.

```
template <typename T>
concept Addable = requires(T a, T b) { { a + b } -> std::convertible_to<T>;
};   // what must compile, and what it must give

template <Addable T>                    // 1. constrained template parameter
T sum2(T a, T b) { return a + b; }

template <typename T>
requires std::integral<T>               // 2. a requires clause
T twice(T v) { return v * 2; }

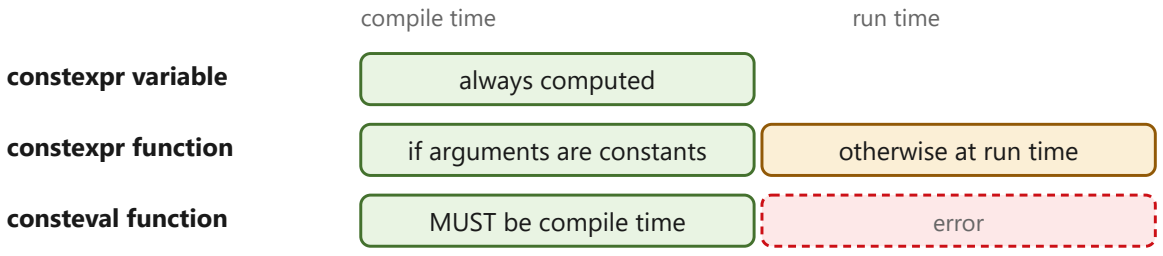
void show(std::integral auto x)       { std::cout << "integral " << x <<
'\n'; }       // 3. constrained auto (abbreviated)
void show(std::floating_point auto x) { std::cout << "floating " << x <<
'\n'; }      //    overloads by concept

static_assert(Addable<int> && Addable<std::string> && !Addable<NoPlus>);
// concepts are compile-time booleans
sum2(2, 3);                              // 5
sum2(std::string("a"), std::string("b"));   // "ab"
twice(21);                               // 42 (compiled)
show(7);  show(2.5);                     // integral 7 / floating 2.5
// twice(2.5);   // error: no matching function, constraint
'std::integral<double>' not satisfied
```

- A **requires-expression** (`requires(T a) { ... }`) lists operations that must compile. `{ expr } -> Concept` also checks the result type.
- The standard library ships many: `std::integral`, `std::floating_point`, `std::same_as`, `std::convertible_to`, `std::invocable`, `std::regular`, and the range concepts such as `std::ranges::range`.
- **Overloads** can differ only by concept; when two are satisfied, the more constrained one wins.
- A concept checks *syntax and types*, not behaviour: `Addable` does not promise that `+` is commutative. Document the meaning you expect.

TOOL	ERROR WHEN IT FAILS	USE FOR
`enable_if` / SFINAE	pages of "no matching function" notes	pre-C++20 code
`static_assert` on a trait	your message, but inside the function body	a single check with a custom message
concept	"constraint not satisfied" at the call site	the interface of a template (C++20 and later)

`constexpr`, `consteval`, `constinit`



constinit: initialised at compile time, still mutable (avoids static-init-order problems).

constexpr does not force compile time; assign to a constexpr variable or use consteval.

Write an ordinary function and ask for compile-time evaluation.

KEYWORD	MEANING
`constexpr` variable	Must be initialised with a constant expression; usable where constants are required (array sizes, template arguments).
`constexpr` function	*Can* run at compile time if its arguments are constants; can still be called at run time with ordinary values.
`consteval` (C++20)	An "immediate" function: every call **must** be evaluated at compile time.
`constinit` (C++20)	The variable is initialised at compile time (no static-initialisation-order problems), but remains mutable.

```
constexpr int factorial(int n) { return n ≤ 1 ? 1 : n * factorial(n - 1); }

constexpr int f10 = factorial(10);        // computed by the compiler
int arr[factorial(4)];                    // array of 24: needs a constant
expression

int n = 5;
int r = factorial(n);                     // fine: runs at run time, because
n is not a constant

consteval int sq(int x) { return x * x; }
constexpr int a = sq(7);                  // OK
// int b = sq(n);                         // ERROR: n is not a constant
expression
```

READ MORE

Effective Modern C++ (Meyers)

- Item 15, Use constexpr whenever possible (PDF pages 115 to 121)

Template metaprogramming (TMP)

Before `constexpr` loops, compile-time computation was **recursive template instantiation**. You will meet it in older code.

```
template <int N>
struct Fact {
    static constexpr int value = Fact<N - 1>::value * N;    // the compiler
instantiates Fact<9>, Fact<8>, ...
};
template <>
struct Fact<0> {                                             // full
specialisation ends the recursion
    static constexpr int value = 1;
};

static_assert(Fact<5>::value == 120);
```

Each `Fact<N>` is a separate class; the result is a baked-in constant.

- **Limits:** recursion depth cap (a few hundred), slow builds, long errors.
- **Today:** `constexpr` for values, traits + `if constexpr`/concepts for types; TMP only to compute *types*.

TASK	OLD STYLE	MODERN STYLE
Compute a value	recursive struct with `::value`	`constexpr` function
Pick code by type	`enable_if` overloads	`if constexpr` or concepts
Reject bad types	obscure substitution errors	`static_assert` / `requires`